

Playwright JavaScript — Top Interview Questions

Core Concepts — Rahul Shetty Academy

A curated set of commonly-asked conceptual Playwright interview questions, with elaborated explanations, syntax, and CLI commands.

Q1. What is the difference between Browser, Context, and Page?

- **Browser** — a single running instance of Chromium/Firefox/WebKit. Launching a browser is expensive, so Playwright reuses one browser across many tests.
- **BrowserContext** — an isolated, incognito-like session inside a browser. Each context has its own cookies, localStorage, and cache — nothing leaks between contexts. This is what makes Playwright tests parallel-safe without needing separate browser installs.
- **Page** — a single tab within a context. A context can have multiple pages (tabs).

JS

```
const browser = await chromium.launch();
const context = await browser.newContext(); // isolated session
const page = await context.newPage();      // a tab
```

Q2. What are the different timeout types in Playwright, and how are they configured?

Playwright has five distinct timeout layers, each solving a different problem:

Timeout type	What it covers	How to set
Test timeout	Total time a single test is allowed to run	test.setTimeout(60000) or config timeout: 60000
Action timeout	Time for a single action (click, fill, check) to become actionable	{ timeout: 5000 } per call, or config use: { actionTimeout }
Navigation timeout	Time for goto, waitForNavigation, etc.	{ timeout: 15000 } per call, or config use: { navigationTimeout }
Assertion timeout	Time expect() retries before failing (web-first assertions)	config expect: { timeout: 10000 }
Global timeout	Time for the entire test run (all tests combined)	config globalTimeout: 3600000

JS

```
// playwright.config.js
export default {
  timeout: 30000,          // per-test
  expect: { timeout: 5000 }, // per-assertion
  use: {
    actionTimeout: 10000,
    navigationTimeout: 30000,
  },
  globalTimeout: 3600000,
};
```

Q3. How does Playwright achieve auto-waiting, and what makes locators "resilient"?

Locators are lazy — they don't resolve to an element until an action is performed, and they re-query the DOM every single time. Before acting, Playwright automatically waits for the element to be attached, visible, stable (not animating), enabled, and receiving events. This eliminates the flaky `Thread.sleep()` / `explicit-wait` patterns common in Selenium.

JS

```
await page.getByRole('button', { name: 'Submit' }).click(); // auto-waits internally
```

Locators are also resilient to DOM re-renders (e.g., React re-mounting a component), whereas `ElementHandle` references can go stale.

Q4. How do you run tests in parallel, and how do you control the number of workers?

Playwright parallelizes at the test file level by default — each file runs in its own worker process. Workers are isolated OS processes (not just threads), so there's no shared state risk.

JS

```
// playwright.config.js
export default { workers: 4 };
```

CLI overrides:

BASH

```
npx playwright test --workers=4      # run with 4 parallel workers
npx playwright test --workers=1      # force serial execution
npx playwright test --workers=50%    # use 50% of available CPU cores
```

To force tests within a single file to run serially (e.g., shared state), use `test.describe.serial()`.

Q5. How do you handle multiple tabs or popup windows?

Listen for the `page` event on the context (new tab) or the `popup` event on the triggering page (`window.open`-style popups), and capture the new `Page` object.

JS

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#open-new-tab'),
]);
await newPage.waitForLoadState();
```

Q6. How do you interact with elements inside an iframe?

Use `frameLocator()`, which scopes all locators to inside the `iframe` and still auto-waits — no manual `switchTo().frame()` needed like in Selenium.

JS

```
await page.frameLocator('#paymentFrame').getByRole('button', { name: 'Pay Now' }).click();
```

Q7. `toBeAttached()` vs `toBeVisible()` — what's the difference?

- **`toBeAttached()`** — element exists in the DOM (could be hidden via CSS, off-screen, `visibility:hidden`, or `zero-opacity`).
- **`toBeVisible()`** — element exists and is actually rendered/visible to a user (has size, no `display:none`, no `visibility:hidden`).

JS

```
await expect(locator).toBeAttached(); // just present in DOM
await expect(locator).toBeVisible(); // present AND visible
```

Common trap: elements can be attached but not visible (e.g., a collapsed accordion panel) — asserting the wrong one causes false positives.

Q8. How does Playwright support API testing and network interception?

Two distinct capabilities:

- **`APIRequestContext`** — pure API testing without a browser at all (fast, ideal for setup/teardown or backend contract tests).
- **`page.route()`** — intercept, modify, or mock network requests made by the browser during UI tests.

JS

```
// Pure API call
const response = await request.get('https://api.example.com/users/1');

// Mocking a network response during a UI test
await page.route('**/api/users', route =>
  route.fulfill({ json: [{ id: 1, name: 'Rahul' }] })
);
```

Q9. What is the Page Object Model (POM) and why is it recommended?

POM encapsulates a page's locators and interactions inside a class, so tests read as business logic rather than raw selectors. This decouples test scripts from UI structure — if a locator changes, you fix it in one place.

JS

```
class LoginPage {
  constructor(page) { this.page = page; }
  async login(username, password) {
    await this.page.getByLabel('Username').fill(username);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Login' }).click();
  }
}
```

Q10. How do you run a specific test file, test, or tag from the command line?

BASH

```
npx playwright test tests/login.spec.js           # run one file
npx playwright test -g "should login successfully" # run by test name (grep)
npx playwright test --grep @smoke                  # run by tag
npx playwright test --grep-invert @slow             # exclude a tag
```

```
npx playwright test --project=chromium # run only on one browser
```

Tags are just conventions in the test title, e.g. `test('login works @smoke', async () => {})`.

Q11. What is the difference between `--headed` and `--debug` mode?

- **`--headed`** — runs the browser with a visible UI instead of headless (useful to visually watch execution).
- **`--debug`** — opens the Playwright Inspector, pauses before each action, and lets you step through the test line by line.

BASH

```
npx playwright test --headed
npx playwright test --debug
npx playwright test tests/login.spec.js --debug # debug a single file
```

Q12. How do you generate and view an HTML test report / trace?

Playwright's trace viewer captures a full timeline (DOM snapshots, network, console, screenshots) for failed (or all) tests — critical for debugging CI failures without reproducing locally.

BASH

```
npx playwright show-report # open last HTML report
npx playwright show-trace trace.zip # open a specific trace file
```

JS

```
// playwright.config.js
use: { trace: 'on-first-retry' } // capture trace only when a test retries after
failing
```

Q13. How does Playwright handle retries, and how do you configure them?

Retries re-run a failed test up to N more times before marking it failed — useful for smoothing over environment flakiness in CI, though it shouldn't be a substitute for fixing genuinely flaky tests.

JS

```
// playwright.config.js
export default { retries: 2 };
```

BASH

```
npx playwright test --retries=2
```

Q14. How do you run tests across multiple browsers/projects?

Projects in the config define browser + settings combinations; `--project` on the CLI filters which ones run.

JS

```
// playwright.config.js
projects: [
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },
]
```

BASH

```
npx playwright test # runs all projects
```

```
npx playwright test --project=firefox # only Firefox
npx playwright test --project=chromium --project=webkit # multiple
```

Q15. How does Playwright handle authentication state reuse across tests?

Instead of logging in through the UI in every test (slow, flaky), Playwright can save browser storage state (cookies + localStorage) once and reuse it — a huge speed win for suites with many authenticated tests.

JS

```
// Save state once (e.g. in a setup project)
await page.context().storageState({ path: 'auth.json' });

// Reuse in config
use: { storageState: 'auth.json' }
```

Q16. What's the difference between soft assertions and regular assertions?

- **Regular expect()** — throws immediately and stops the test on failure.
- **expect.soft()** — logs the failure but lets the test continue, so you can see all failures in one run instead of stopping at the first.

JS

```
await expect.soft(page.getByText('Welcome')).toBeVisible();
await expect.soft(page.getByText('Dashboard')).toBeVisible();
// test continues even if the first soft assertion fails; final result still reflects it
```

Q17. How do you run only failed tests from the last run?

BASH

```
npx playwright test --last-failed
```

Useful in CI-debug loops or after fixing a batch of flaky failures — avoids re-running the entire (often slow) suite.

Q18. How do you update visual snapshots / handle visual regression testing?

Playwright supports pixel-level screenshot comparison via `toHaveScreenshot()`. Baselines are stored per-OS/browser since rendering differs subtly across platforms.

JS

```
await expect(page).toHaveScreenshot('homepage.png');
```

BASH

```
npx playwright test --update-snapshots # regenerate baseline images after an intentional UI change
```

Quick CLI Reference

BASH

```
npx playwright show-report # view HTML report
npx playwright show-trace trace.zip # inspect a trace
npx playwright codegen https://example.com # record actions, generate code
```

Command	Description
<code>npx playwright test</code>	run all tests
<code>npx playwright test --headed</code>	visible browser
<code>npx playwright test --debug</code>	step-through debugger
<code>npx playwright test --workers=4</code>	parallel workers
<code>npx playwright test --project=chromium</code>	specific browser
<code>npx playwright test -g "login"</code>	run tests matching name
<code>npx playwright test --retries=2</code>	retry failed tests
<code>npx playwright test --last-failed</code>	re-run only failures
<code>npx playwright test --update-snapshots</code>	refresh visual baselines